21

Creating and Manipulating Tables

In this chapter, you'll learn the basics of table creation, alteration, and deletion.

Creating Tables

MySQL statements are not used just for table data manipulation. Indeed, MySQL can be used to perform all database and table operations, including to create and manipulate tables.

There are generally two ways to create database tables:

- You can use an administration tool (like the ones discussed in Chapter 2, "Introducing MySQL") to create and manage database tables interactively.
- You can manipulate tables directly with MySQL statements.

To create tables programmatically, you use the `CREATE TABLE` SQL statement. It is worth noting that when you use interactive tools, you are actually using MySQL statements. Instead of writing these statements yourself, however, you have the MySQL Workbench interface generate and execute the MySQL for you.

Note

Additional Examples For additional examples of table creation scripts, see the code used to create the sample tables used in this book, as explained in Appendix B.

Basic Table Creation

To create a table using `CREATE TABLE`, you must specify the following information:

- The name of the new table, which you specify after the `CREATE TABLE` statement
- The name and definition of the table columns, separated by commas

The CREATE TABLE statement can also include other keywords and options, but at a minimum, you need the table name and column details. The following MySQL statement creates the customers table used throughout this book:

Input

```
CREATE TABLE customers
(
  cust_id      int       NOT NULL AUTO_INCREMENT,
  cust_name    char(50)  NOT NULL ,
  cust_address char(50)  NULL ,
  cust_city    char(50)  NULL ,
  cust_state   char(5)   NULL ,
  cust_zip     char(10)  NULL ,
  cust_country char(50)  NULL ,
  cust_contact char(50)  NULL ,
  cust_email   char(255) NULL ,
  PRIMARY KEY (cust_id)
) ENGINE=InnoDB;
```

Analysis

As you can see in this statement, the table name is specified immediately following the CREATE TABLE statement. The actual table definition (all the columns) is enclosed within parentheses. The columns are separated by commas. This particular table is made up of nine columns. Each column definition starts with the column name (which must be unique within the table), followed by the column's datatype. (Refer to Chapter 1, "Understanding SQL," for an explanation of datatypes. In addition, Appendix D, "MySQL Datatypes," lists the datatypes supported by MySQL.) The table's primary key may be specified at table creation time using the PRIMARY KEY keyword. Here, the column cust_id is specified as the primary key column. The entire statement is terminated with a semicolon after the closing parenthesis. (Ignore the ENGINE=InnoDB and AUTO_INCREMENT statements for now; we'll come back to them later.)

Tip

Statement Formatting As you will recall, white space is ignored in MySQL statements. You can type a statement on one long line or break it up over many lines. It makes no difference which way you do it. This flexibility enables you to format your SQL as best suits you. The preceding CREATE TABLE statement is a good example of MySQL statement formatting; the code is specified over multiple lines, with the column definitions indented for ease of reading and editing. Formatting your MySQL in this way is entirely optional but highly recommended.

> **Tip**
>
> **Handling Existing Tables** When you create a new table, the table name specified must not already exist, or you'll generate an error. To prevent accidental overwriting, SQL requires that you first manually remove a table (see later sections for details) and then re-create it rather than just overwrite it.
>
> If you want to be sure you're creating a table that does not already exist, specify `IF NOT EXISTS` after the table name. MySQL does not check to see that the schema of the existing table matches the one you are about to create. It simply checks to see if the table name exists, and it proceeds with table creation only if it does not.

Working with NULL Values

Back in Chapter 6, "Filtering Data," you learned that a `NULL` value is no value or the lack of a value. A column that allows `NULL` values also allows rows to be inserted with no value at all in that column. A column that does not allow `NULL` values does not accept rows with no value; in other words, that column will always be required when rows are inserted or updated.

Every table column is either a `NULL` column or a `NOT NULL` column, and that state is specified in the table definition at creation time. Take a look at the following example:

Input

```
CREATE TABLE orders
(
  order_num  int      NOT NULL AUTO_INCREMENT,
  order_date datetime NOT NULL ,
  cust_id    int      NOT NULL ,
  PRIMARY KEY (order_num)
) ENGINE=InnoDB;
```

Analysis

This statement creates the `orders` table used throughout this book. `orders` contains three columns: one for the order number, one for the order date, and one for the customer ID. All three columns are required, and so each contains the keyword `NOT NULL` to prevent the insertion of columns with no value. If someone tries to insert no value, an error will be returned, and the insertion will fail.

This next example creates a table with a mixture of `NULL` and `NOT NULL` columns:

Input

```
CREATE TABLE vendors
(
  vend_id      int NOT  NULL AUTO_INCREMENT,
  vend_name    char(50) NOT NULL ,
  vend_address char(50) NULL ,
  vend_city    char(50) NULL ,
```

```
  vend_state   char(5)  NULL ,
  vend_zip     char(10) NULL ,
  vend_country char(50) NULL ,
  PRIMARY KEY (vend_id)
) ENGINE=InnoDB;
```

► Analysis

This statement creates the `vendors` table used throughout this book. The vendor ID and vendor name columns are both required, and are, therefore, specified as `NOT NULL`. The five remaining columns all allow `NULL` values, and so `NOT NULL` is not specified. `NULL` is the default setting, so if `NOT NULL` is not specified, `NULL` is assumed.

Caution

Understanding `NULL` Don't confuse `NULL` values with empty strings. A `NULL` value is the lack of a value; it is not an empty string. You could, for example, specify `''` (two single quotes with nothing in between them) in a `NOT NULL` column because an empty string is a valid value; it is not no value. `NULL` values are specified with the keyword `NULL`, not with an empty string.

Primary Keys Revisited

As explained earlier, primary key values must be unique. That is, every row in a table must have a unique primary key value. If a single column is used for the primary key, it must be unique; if multiple columns are used, the combination of those columns must be unique.

The `CREATE TABLE` examples shown thus far use a single column as the primary key. The primary key is defined using a statement such as:

```
PRIMARY KEY (vend_id)
```

To create a primary key made up of multiple columns, simply specify the column names as a comma-delimited list, as shown in this example:

```
CREATE TABLE orderitems
(
  order_num  int          NOT NULL ,
  order_item int          NOT NULL ,
  prod_id    char(10)     NOT NULL ,
  quantity   int          NOT NULL ,
  item_price decimal(8,2) NOT NULL ,
  PRIMARY KEY (order_num, order_item)
) ENGINE=InnoDB;
```

The `orderitems` table contains the order specifics for each order in the `orders` table. There may be multiple items per order, but each order will only ever have one first item, one second item, and so on. As such, the combination of order number

(column order_num) and order item (column order_item) are unique, and thus the combination is suitable to be the primary key, which is defined as follows:

```
PRIMARY KEY (order_num, order_item)
```

Primary keys may be defined at table creation time (as shown here) or after table creation (as discussed later in this chapter).

Tip

No Defining Primary Keys with NULL Values Back in Chapter 1, you learned that primary keys are columns whose values uniquely identify every row in a table. Only columns that do not allow NULL values can be used in primary keys. Columns that allow no value at all cannot be used as unique identifiers.

Using AUTO_INCREMENT

Let's take a look at the customers and orders tables again. Customers in the customers table are uniquely identified by the column cust_id, which includes a unique number for each and every customer. Similarly, orders in the orders table each have a unique order number, which is stored in the column order_num. These numbers have no special significance other than the fact that they are unique. When a new customer or order is added, a new customer ID or order number is needed. The numbers can be anything, as long as they are unique.

Obviously, the simplest number to use would be whatever comes next—whatever is one higher than the current highest number. For example, if the highest cust_id is 10005, the next customer inserted into the table could have the cust_id 10006.

Simple, right? Well, not really. How would you determine the next number to be used? You could, of course, use a SELECT statement to get the highest number (using the Max() function introduced in Chapter 12, "Summarizing Data") and then add 1 to it. But that would not be safe, as you'd need to find a way to ensure that no one else inserted a row in between the time that you performed the SELECT and the INSERT, which is a legitimate possibility in multiuser applications. It also wouldn't be efficient (as performing additional MySQL operations is never ideal).

This is where AUTO_INCREMENT comes in. Look at the following line, which is part of the CREATE TABLE statement used to create the customers table:

```
cust_id      int       NOT NULL AUTO_INCREMENT
```

AUTO_INCREMENT tells MySQL that this column is to be automatically incremented each time a row is added. Each time an INSERT operation is performed, MySQL automatically increments (hence the name AUTO_INCREMENT) the column, assigning it the next available value. This way, each row is assigned a unique cust_id, which is then used as the primary key value.

Only one AUTO_INCREMENT column is allowed per table, and it must be indexed (for example, by being made a primary key).

Note

Overriding `AUTO_INCREMENT` Need to use a specific value in a column designated as `AUTO_INCREMENT`? You can. Simply specify a value in the `INSERT` statement, and as long as it is unique (that is, has not been used yet), that value will be used instead of an automatically generated one. Subsequent incrementing will start using the value manually inserted. (See the table population scripts in Appendix B for examples of this.)

Tip

Determining the `AUTO_INCREMENT` Value One downside of having MySQL generate primary keys for you (via `AUTO_INCREMENT`) is that you don't know what those values are.

Consider this scenario: You are adding a new order. This requires creating a single row in the `orders` table and then a row for each item ordered in the `orderitems` table. The order number is stored, along with the order details, in `orderitems`. This is how the `orders` and `orderitems` table are related to each other. And this obviously requires that you know the generated `order_num` after the order's row is inserted and before the `orderitems` rows are inserted.

So how could you obtain this value when an `AUTO_INCREMENT` column is used? By using the `last_insert_id()` function, like this:

```
SELECT last_insert_id();
```

This returns the last `AUTO_INCREMENT` value, which you can then use in subsequent MySQL statements.

Specifying Default Values

MySQL enables you to specify default values to be used if no value is specified when a row is inserted. Default values are specified using the `DEFAULT` keyword in the column definitions in the `CREATE TABLE` statement.

Look at the following example:

Input

```
CREATE TABLE orderitems
(
  order_num  int          NOT NULL ,
  order_item int          NOT NULL ,
  prod_id    char(10)     NOT NULL ,
  quantity   int          NOT NULL DEFAULT 1,
  item_price decimal(8,2) NOT NULL ,
  PRIMARY KEY (order_num, order_item)
) ENGINE=InnoDB;
```

Analysis

This statement creates the `orderitems` table, which contains the individual items that make up an order. (The order itself is stored in the `orders` table.) The `quantity` column contains the quantity for each item in an order. In this example, adding the text `DEFAULT 1` to the column description instructs MySQL to use a quantity of 1 if no quantity is specified.

Caution

Functions Are Not Allowed Unlike most DBMSs, MySQL does not allow the use of functions as `DEFAULT` values; only constants are supported.

Tip

Using `DEFAULT` Instead of `NULL` Values Many database developers use `DEFAULT` values instead of `NULL` columns, especially in columns that will be used in calculations or data groupings.

Engine Types

You might have noticed that the `CREATE TABLE` statements used thus far have all ended with an `ENGINE=InnoDB` statement.

Like every other DBMS, MySQL has an internal engine that actually manages and manipulates data. When you use the `CREATE TABLE` statement, that internal engine is used to actually create the tables, and when you use the `SELECT` statement or perform any other database processing, the engine is used to process your request. For the most part, the engine is buried within the DBMS, and you need not pay much attention to it.

But unlike every other DBMS, MySQL does not come with a single engine. Rather, it ships with several engines, all buried within the MySQL server and all capable of executing commands such as `CREATE TABLE` and `SELECT`.

So why bother shipping multiple engines? Because they each have different capabilities and features, and being able to pick the right engine for the job gives you unprecedented power and flexibility.

Of course, you are free to totally ignore database engines. If you omit the `ENGINE=` statement, the default engine is used (most likely MyISAM), and most of your SQL statements will work as is. But not all of them will, and that is why this is important (and why two engines are used in the sample tables used in this book).

Here are several engines to be aware of:

- InnoDB is a transaction-safe engine (see Chapter 26, "Managing Transaction Processing"). It does not support full-text searching.
- MEMORY is functionally equivalent to MyISAM, but data is stored in memory (instead of on disk), so it is extremely fast (and ideally suited for temporary tables).
- MyISAM is a very high-performance engine. It supports full-text searching (see Chapter 18, "Full-Text Searching") but does not support transactional processing.

Note

To Learn More For a complete list of supported engines, see the MySQL documentation.

Engine types may be mixed. The example tables used throughout this book all use InnoDB with the exception of the `productnotes` table, which uses MyISAM. The reason for this is that I wanted support for transactional processing (and thus used InnoDB) but also needed full-text searching support in `productnotes` (and thus used MyISAM for that table).

Caution

Foreign Keys Can't Span Engines There is one big downside to mixing engine types. Foreign keys (used to enforce referential integrity, as explained in Chapter 1) cannot span engines. That is, a table using one engine cannot have a foreign key that refers to a table that uses another engine.

So, which engine should you use? Well, it depends on what features you need. MyISAM tends to be the most popular engine because of its performance and features. But if you do need transaction-safe processing, you will need to use a different engine.

Updating Tables

To update table definitions, you use the `ALTER TABLE` statement. However, ideally, the design of a table should never be altered once the table contains data. You should spend sufficient time anticipating future needs during the table design process so that extensive changes are not required later on.

When you simply must change a table, you can do so by using `ALTER TABLE`. You must specify the following information with it:

- The name of the table to be altered after the keywords `ALTER TABLE` (The table you specify must exist, or an error will be generated.)
- The list of changes to be made

The following example adds a column to a table:

Input

```
ALTER TABLE vendors
ADD vend_phone CHAR(20);
```

Analysis

This statement adds a column named `vend_phone` to the `vendors` table. The datatype must be specified.

To remove this newly added column, you can use the following:

Input

```
ALTER TABLE Vendors
DROP COLUMN vend_phone;
```

One common use for `ALTER TABLE` is to define foreign keys. The following is the code used to define the foreign keys used by the tables in this book:

```
ALTER TABLE orderitems
ADD CONSTRAINT fk_orderitems_orders
FOREIGN KEY (order_num) REFERENCES orders (order_num);

ALTER TABLE orderitems
ADD CONSTRAINT fk_orderitems_products FOREIGN KEY (prod_id) REFERENCES products
  (prod_id);

ALTER TABLE orders
ADD CONSTRAINT fk_orders_customers FOREIGN KEY (cust_id) REFERENCES customers
  (cust_id);

ALTER TABLE products
ADD CONSTRAINT fk_products_vendors
FOREIGN KEY (vend_id) REFERENCES vendors (vend_id);
```

Four `ALTER TABLE` statements are used here because four different tables are being altered. To make multiple alterations to a single table, you can use a single `ALTER TABLE` statement and list the alterations, separated by commas.

Complex table structure changes usually require a manual move process involving these steps:

1. Create a new table with the new column layout.
2. Use the `INSERT SELECT` statement to copy the data from the old table to the new table. (See Chapter 19, "Inserting Data," for details on the `INSERT SELECT` statement.) Use conversion functions and calculated fields, if needed.
3. Verify that the new table contains the desired data.
4. Rename the old table (or delete it, if you are really brave).
5. Rename the new table with the name previously used by the old table.
6. Re-create any triggers, stored procedures, indexes, and foreign keys, as needed.

Caution

Use `ALTER TABLE` Carefully Use `ALTER TABLE` with extreme caution and be sure you have a complete set of backups (both schema and data) before proceeding. Database table changes cannot be undone—and if you add columns you don't need, you might not be able to remove them. Similarly, if you drop a column that you do need, you might lose all the data in that column.

Deleting Tables

Deleting a table (that is, actually removing the entire table and not just the contents) is very easy—arguably too easy. You deleted a table by using the `DROP TABLE` statement:

Input

```
DROP TABLE customers2;
```

Analysis

This statement deletes the `customers2` table (assuming that it exists). You get no confirmation, and there is no undo. When you execute this statement, MySQL permanently removes the table.

Renaming Tables

To rename a table, use the `RENAME TABLE` statement as follows:

Input

```
RENAME TABLE customers2 TO customers;
```

Analysis

`RENAME TABLE` does just what it says it does: It renames a table. You can rename multiple tables in one operation like this:

```
RENAME TABLE backup_customers TO customers,
             backup_vendors TO vendors,
             backup_products TO products;
```

Summary

In this chapter, you learned several new SQL statements. `CREATE TABLE` is used to create new tables, `ALTER TABLE` is used to change table columns (or other objects, like constraints or indexes), and `DROP TABLE` is used to completely delete a table. These statements should be used with extreme caution—and only after backups have been made. You also learned about database engines, defining primary and foreign keys, and other important table and column options.

Challenges

1. Add a website column (`vend_web`) to the `Vendors` table. You need a text field big enough to accommodate a URL.
2. Use `UPDATE` statements to update `Vendors` table records to include a website. (You can make up any address.)